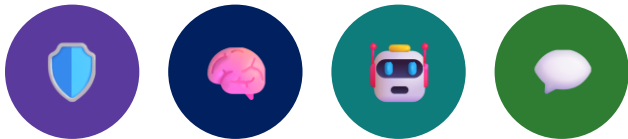


"AI can draft the first pass -- only you can be accountable for what ships."

AI THREADS

AI Threads Six Weeks of Disciplined Practice

A short, recurring feature woven after every week of the bootcamp -- not extra modules, but a running discipline in how we use AI coding assistants responsibly, verify what they produce, and defend every AI-assisted decision we ship.





AI THREADS OVERVIEW

Six compact AI-focused threads, one woven after each week of the bootcamp -- not extra modules, but a running discipline.

Each thread covers AI best practices and governance, this week's technical impact, the agentic tools in use, and closes with a classroom discussion prompt.

WEEKS 1-3



Week 1 — AI in Java Fundamentals

Using AI to explain the JVM and Java idioms -- and why beginners must verify every explanation against the JLS and real docs.



Week 2 — AI in Backend Development and Testing

Copilot goes formal this week: AI-assisted endpoints, tests, and logging -- plus the discipline to catch shallow, tautological AI-written tests.



Week 3 — AI in Enterprise Frameworks

AI-assisted Spring wiring and configuration, with mandatory human review of anything AI proposes for Spring Security.

WEEKS 4-6



Week 4 — AI Across the Full Stack

AI-assisted Angular components, SQL, and Kafka event schemas -- and the risk of a query that 'looks right' but isn't indexed.



Week 5 — AI in DevOps and Delivery

AI-drafted Terraform, Ansible, and CI/CD pipelines -- never applied without a human reviewing every line first.



Week 6 — AI in the Capstone

Using AI responsibly across a real deliverable, with an AI usage log and the ability to defend every AI-assisted decision.

KEY TAKEAWAY AI Threads run six times, not once -- the same governance, impact, and tools lenses applied to a new domain each week, always closing with a discussion the class owns.

WEEK 1 AI THREAD

AI in Java Fundamentals

THIS WEEK'S DOMAIN

- Week 1 covers JVM architecture and the runtime model, core Java syntax, object-oriented design, memory management and garbage collection, the Collections Framework, streams and functional programming, and exception handling -- the vocabulary every later week builds on.
- This week's own modules are entirely lecture-and-lab, with no AI tooling formally taught yet -- GitHub Copilot arrives next week, which is exactly why this thread's focus is judgment, not workflow.

WHY AN AI THREAD NOW

- This is most students' first heavy exposure to an AI coding assistant explaining Java itself. A confident, fluent, wrong explanation of the JVM or garbage collection is far more dangerous here than a bug in generated code -- it can quietly become the foundation students reason from for the rest of the bootcamp.
- The risk compounds silently: a wrong mental model formed here can distort how a student reasons about performance and memory for every week that follows.

WHAT'S AHEAD IN THIS THREAD

- Four short stops: responsible use of AI for learning Java fundamentals, AI's real impact on how beginners study the language, the agentic tools already available to you, and a discussion question to close out Week 1.

KEY TAKEAWAY Treat an AI assistant as a fast first draft of an explanation, never as the final word on how the JVM, GC, or Java syntax actually behaves -- the JLS and the official docs are the final word.

AI BEST PRACTICES AND GOVERNANCE

Ground rules for using an AI assistant to learn Java fundamentals, from day one of the bootcamp.



Verify Before You Believe

Cross-check any AI explanation of the JVM, GC, or Java syntax against the Java Language Specification or the official Oracle docs before you repeat it.



Never Paste Secrets or Real Data

Don't paste API keys, credentials, or real personal data into a prompt to 'explain this code' -- use synthetic examples instead, from day one.



AI Explains, You Understand

If you can't re-explain a JVM concept in your own words after the AI explains it, you don't understand it yet -- ask a follow-up, don't just copy the answer.



Keep a Light Usage Habit

Note when and how you used AI for a fundamentals exercise -- a one-line habit now becomes the AI usage log the capstone will require in Week 6.

KEY TAKEAWAY An AI assistant is a study partner for Java fundamentals, not an authority -- verify its claims about the JVM and language behavior against real documentation every time.

AI'S IMPACT ON JAVA FUNDAMENTALS

How an AI assistant changes the experience of learning the JVM, syntax, and core language mechanics.

IMPACT AREA

Area	What Changes This Week
Learning the JVM	AI narrates GC phases well but can hallucinate details -- always confirm against the JLS.
Reading unfamiliar syntax	AI reliably explains an unfamiliar construct, like a stream pipeline, line by line.
Debugging collections and streams	Good at spotting an off-by-one, but can misdiagnose a subtle concurrency bug -- test the fix.
Exception handling habits	AI examples often over-catch broad exceptions; learn the narrow, correct pattern instead.

BOTTOM LINE

AI accelerates exposure to Java concepts, but fundamentals week is exactly the wrong week to skip verifying what it tells you -- you don't yet have the experience to catch a wrong answer on instinct.

KEY TAKEAWAY AI speeds up how fast you encounter Java concepts; it does not replace the work of verifying and internalizing them.

AGENTIC TOOLS IN USE

The AI tools available to you this week, before GitHub Copilot is formally introduced in Week 2.



Chat-Based AI Assistants

General assistants are useful this week for asking 'explain this JVM concept' or 'why does this stream throw' -- treat answers as a starting point.



IDE-Integrated Suggestions

If your IDE already offers inline AI code completion, notice it this week without leaning on it -- Week 2 formally teaches GitHub Copilot as a workflow, not just autocomplete.



AI-Assisted Documentation Reading

Ask an assistant to summarize a dense JLS or Javadoc page, then go read the actual page -- the summary is a map, not the territory.



Human Instructor as the Tiebreaker

When an AI explanation and your instructor's explanation disagree, the instructor and the official docs win -- say so explicitly and ask why.

KEY TAKEAWAY This week's agentic tools are lightweight and exploratory -- the deliberate, workflow-level use of AI tooling begins formally in Week 2 with GitHub Copilot.

DISCUSSION PROMPT

A short classroom conversation to close out Week 1's AI thread.

DISCUSSION QUESTIONS

- 1) An AI assistant just gave you a confident, detailed explanation of how the JVM garbage collector works -- and it's subtly wrong. How would you have caught that, given what you know this week?
- 2) Should a fundamentals-week exercise ever be 'explained' entirely by AI, with no time spent reading the JLS or Javadoc directly? Why or why not?

WHY IT'S WORTH DEBATING

- Students are forming their study habits for the rest of the bootcamp this week. If 'ask the AI and move on' becomes the default now, it compounds across five more weeks of increasingly complex material.

FACILITATION NOTE

- Ask for a show of hands on who has already caught an AI assistant giving a wrong explanation this week -- use a real example from the room if one comes up, it lands harder than a hypothetical.

KEY TAKEAWAY The habit you build in Week 1 -- verify, don't just accept -- is the same habit that keeps AI-assisted work safe in every later week.

WEEK 2 AI THREAD

AI in Backend Development and Testing

THIS WEEK'S DOMAIN

- Week 2 moves from language fundamentals into real backend engineering: Maven and project structure, GitHub Copilot as a formal tool, REST API design, DTOs and validation, service-layer design, JUnit and Mockito, and logging and observability.
- This is also the first week students write real automated tests, which is exactly where an AI assistant's speed can outpace its judgment.

WHY AN AI THREAD NOW

- This is the week Copilot stops being optional exploration and becomes a taught workflow. It's also the week students start writing tests -- exactly where AI's habit of generating shallow, tautological tests can quietly erode the value of a whole test suite if nobody is watching for it.
- A shallow test that passes gives false confidence at exactly the moment a codebase starts growing past what one person can manually re-verify by hand.

WHAT'S AHEAD IN THIS THREAD

- Best practices for Copilot-assisted backend work, AI's real impact on API and test-writing speed, the agentic tools students will use hands-on this week, and a discussion question on trusting AI-generated tests.

KEY TAKEAWAY Copilot goes from novelty to workflow this week -- the discipline you build now for reviewing its suggestions is the discipline that scales to every backend module after this one.

AI BEST PRACTICES AND GOVERNANCE

Ground rules for the week Copilot becomes a real part of the backend workflow.



Review Every Suggestion Before Accepting

Tab-accepting a Copilot suggestion without reading it is the single most common way AI-introduced bugs reach a codebase -- read it like you'd read a teammate's PR.



Keep Secrets Out of Prompts and Repos

Never paste real API keys, database credentials, or .env contents into a Copilot chat or comment prompt -- Copilot suggestions are also context, and context can leak.



Verify Against Real Spring/Java Docs

Copilot can suggest an outdated annotation or a deprecated Mockito API -- confirm against the current Spring Boot and JUnit 5 documentation, not just what looks familiar.



A Human Still Owns the Merge

An AI-assisted PR still needs a human reviewer who read the diff, not just someone who ran the tests and saw green -- code review discipline doesn't relax because AI wrote the first draft.

KEY TAKEAWAY GitHub Copilot writes suggestions, not decisions -- every accepted suggestion is still your code, and your review responsibility.

AI'S IMPACT ON BACKEND DEVELOPMENT AND TESTING

How Copilot and AI-assisted workflows change REST API, service-layer, and test work this week.

IMPACT AREA

Area	What Changes This Week
REST endpoint scaffolding	Fast at drafting a controller and DTO -- but check validation annotations and status codes.
Unit test generation	Often shallow -- asserts 'no exception thrown' instead of a real outcome. Read every assertion.
Service-layer design	Plausible service methods, but DTO/entity boundaries and transaction strategy stay yours.
Logging and observability	Genuinely useful for structured log statements and correlation-id boilerplate.

BOTTOM LINE

The productivity win is real for scaffolding and logging; the risk is concentrated in tests -- a green test suite full of shallow assertions is worse than no tests, because it looks like safety.

KEY TAKEAWAY A passing AI-generated test proves the code ran, not that it's correct -- check what the test actually asserts, not just whether it's green.



AGENTIC TOOLS IN USE

The tools this week's modules put directly in your hands.



GitHub Copilot

Formally taught this week for inline suggestions, chat-based scaffolding, and Copilot-assisted refactoring across Java and Spring code.



AI-Assisted Test Generation

Using Copilot or a chat assistant to draft a starting JUnit/Mockito test, then rewriting the assertions yourself to check real behavior.



AI-Assisted Code Review

Asking an assistant to review a diff for obvious issues before opening a PR -- a second pass, not a replacement for a human reviewer.



AI-Assisted Log/Observability Drafting

Using AI to draft consistent structured-logging statements and correlation-id handling across a service, then verifying the fields actually match your logging standard.

KEY TAKEAWAY This week's tools multiply how much backend code you can produce -- code review discipline is what keeps that a benefit instead of a liability.

DISCUSSION PROMPT

A short classroom conversation to close out Week 2's AI thread.

DISCUSSION QUESTIONS

- 1) Copilot generates a JUnit test that passes but only asserts that no exception was thrown. Is that test 'done'? What's missing, and whose job is it to catch it?
- 2) If Copilot drafts 80% of a service method correctly but gets the transaction boundary wrong, what does that say about how you should read AI suggestions for business-critical code?

WHY IT'S WORTH DEBATING

- Students are about to rely on Copilot heavily for the rest of the bootcamp. Getting concrete about what 'a passing test' actually proves -- this week, before habits set -- prevents a semester of shallow test suites.

FACILITATION NOTE

- Have a volunteer share a Copilot-generated test from their own lab work on screen and ask the class to critique what it actually verifies -- real code beats a hypothetical every time.

KEY TAKEAWAY A test suite is only as trustworthy as its assertions -- 'AI wrote it and it's green' is not the same as 'AI wrote it and it's correct.'

WEEK 3 AI THREAD

AI in Enterprise Frameworks

THIS WEEK'S DOMAIN

- Week 3 goes deep into the Spring ecosystem: Spring Core and dependency injection, Spring Boot auto-configuration, service and repository layers, configuration and profiles, transaction management, Spring Security, and validation with global exception handling.
- Spring Security specifically introduces the first genuinely production-grade security surface students configure themselves in this bootcamp.

WHY AN AI THREAD NOW

- Spring Security is the single highest-stakes place in the whole bootcamp for AI to quietly cause harm -- a syntactically valid, AI-generated security configuration can be dangerously permissive and still look correct to someone skimming it.
- A permissive CORS or CSRF setting doesn't throw an error -- it just quietly works, which is exactly what makes it easy to miss in a quick review.

WHAT'S AHEAD IN THIS THREAD

- AI-assisted Spring wiring and configuration, the real security risk in AI-suggested config, the agentic tools relevant to enterprise-framework work, and a discussion question built around a real insecure-CORS scenario.

KEY TAKEAWAY Spring Security config is the one place this week where 'it compiled and the request worked' is not evidence of correctness -- insecure and working look identical from the outside.

AI BEST PRACTICES AND GOVERNANCE

Why AI-suggested Spring configuration needs a human security reviewer, every time.



Never Auto-Accept Security Config

An AI-suggested SecurityFilterChain, CORS policy, or CSRF setting must be read and understood line by line before it merges -- this is not optional, even under lab time pressure.



Watch for 'Just Make It Work' Suggestions

AI assistants asked to fix an auth error will sometimes suggest disabling CSRF or permitting all origins -- that silences the error without addressing the actual problem.



Cross-Check Against Spring Security Docs

Spring Security's API changes across versions; verify any AI-suggested annotation or bean configuration against the Spring Security docs for the version this course actually uses.



Human Sign-Off on Anything Auth-Related

Any AI-assisted change touching authentication, authorization, or CORS should get an explicit second pair of human eyes before merge -- treat it like a production security review, because it is one.

KEY TAKEAWAY An AI-generated Spring Security configuration that 'looks fine' and 'is secure' are two different claims -- only a human security review can confirm the second one.

AI'S IMPACT ON ENTERPRISE FRAMEWORKS

Where AI genuinely helps with Spring, and where it needs the closest supervision.

IMPACT AREA

Area	What Changes This Week
Bean wiring and DI	Reliable at constructor-injection boilerplate and spotting a missing @Service annotation.
Spring Boot auto-configuration	Speeds up debugging config issues -- verify the explanation against real startup logs.
Spring Security configuration	Highest risk this week -- can suggest permissive CORS/CSRF. Mandatory human review applies.
Transaction boundaries	Can suggest where @Transactional goes, but propagation and isolation need real reasoning.

BOTTOM LINE

Treat AI's help with Spring Security and transaction boundaries as a draft to interrogate, not a fix to accept -- everywhere else in the framework, AI's suggestions are lower-stakes and easier to trust after a quick read.

KEY TAKEAWAY The more a Spring change touches security or data consistency, the less appropriate it is to accept an AI suggestion without deliberately reasoning through it yourself.

AGENTIC TOOLS IN USE

How this week's tools help with Spring -- and where they hand off to a human reviewer.



Copilot for Bean and Config Scaffolding

Drafting @Configuration classes, profile-specific YAML, and repository interfaces quickly, reviewed the same way as any other Copilot suggestion.



Agentic Assistants for Multi-File Reasoning

A tool like Claude Code or Cursor can trace a bean-wiring problem across multiple files faster than manual search -- useful for diagnosis, not for blindly applying its proposed fix.



AI-Assisted Security Config Review

Asking an assistant to explain what a given SecurityFilterChain actually permits is a good sanity check -- but it supplements a human security review, it never replaces one.



AI-Assisted Transaction Reasoning

Using AI to talk through propagation and isolation options for a transaction boundary, then verifying the choice against the actual consistency requirement of the operation.

KEY TAKEAWAY Agentic tools are excellent at multi-file diagnosis in a Spring codebase -- the decision of what a fix should actually do stays with the engineer, especially around security.

DISCUSSION PROMPT

A short classroom conversation to close out Week 3's AI thread.

DISCUSSION QUESTIONS

- 1) An AI assistant "fixes" a CORS error by suggesting `allowedOrigins("*")` with credentials enabled. It works. What's actually wrong with this fix, and how would you catch it in a code review?
- 2) Should security-related code ever be merged on the strength of 'the AI suggested it and it compiled,' even under a lab deadline?

WHY IT'S WORTH DEBATING

- This is a realistic, common failure mode -- AI assistants optimize for making the error go away, not for the security implications of how. Naming the specific anti-pattern now makes it recognizable later, in the capstone and beyond.

FACILITATION NOTE

- Put the `allowedOrigins("*")` plus credentials example on the board and ask students to explain, in their own words, exactly what an attacker could now do -- concreteness beats abstraction here.

KEY TAKEAWAY 'It compiled and the error went away' is never sufficient evidence that an AI-suggested security fix is actually safe.

WEEK 4 AI THREAD

AI Across the Full Stack

THIS WEEK'S DOMAIN

- Week 4 spans the whole stack: Kafka event-driven architecture and resilience patterns, Angular components, state management and frontend-to-API integration, and PostgreSQL, SQL, and Spring Data JPA.
- Few other weeks touch this many distinct technologies -- and distinct technologies mean distinct failure modes AI can quietly introduce.

WHY AN AI THREAD NOW

- This is the widest single week of the bootcamp, technology-wise -- and AI's most dangerous failure mode here is a SQL query that 'looks right,' returns correct results in a small dev database, and is quietly unindexed or unsafe at real scale.
- The common thread across Angular, SQL, and Kafka this week is that a mistake in any of them can pass every functional test and still be wrong.

WHAT'S AHEAD IN THIS THREAD

- Best practices for AI-assisted Angular and SQL work, AI's real impact across frontend, data, and messaging, the agentic tools in play this week, and a discussion question on trusting an AI-suggested query.

KEY TAKEAWAY Across four different technologies in one week, the same rule holds: AI can produce code that runs correctly and still performs or fails badly -- runs correctly is not the same as is correct.

AI BEST PRACTICES AND GOVERNANCE

Full-stack ground rules -- from Angular components to raw SQL.



Review Angular/RxJS Suggestions for Leaks

AI-suggested Angular components and RxJS subscriptions can omit unsubscribe logic or misuse Signals -- check for memory leaks and stale state, not just that the component renders.



Never Trust a Query Plan You Haven't Read

An AI-suggested SQL query can return correct results on a small dataset while missing an index that makes it fall over at real scale -
- run EXPLAIN yourself before trusting it.



Keep Real Data Out of Prompts

Don't paste real customer rows, connection strings, or production schema dumps into an AI prompt when asking for query help -- use the course's synthetic fixtures instead.



Version Event Schemas Deliberately

If AI suggests a Kafka event schema, treat versioning and backward compatibility as a deliberate human decision -- an AI assistant doesn't know your consumers' compatibility guarantees.

KEY TAKEAWAY Across Angular, SQL, and Kafka alike, the question to ask an AI suggestion is not 'does this run' but 'does this hold up under real load, real state changes, and real consumers.'

AI'S IMPACT ACROSS THE FULL STACK

How AI changes work differently across Angular, PostgreSQL, and Kafka this week.

IMPACT AREA

Area	What Changes This Week
Angular component scaffolding	Fast, reliable component and RxJS/Signals boilerplate -- genuine time savings, low risk.
SQL query writing	Can return correct rows while missing an index -- always check the query plan yourself.
Query optimization	A good hypothesis, not a guarantee -- verify with EXPLAIN ANALYZE before and after.
Kafka event-schema design	Proposes a plausible shape quickly; versioning and compatibility need a human decision.

BOTTOM LINE

The frontend is the lowest-risk place for AI assistance this week; the database and event layer are the highest-risk, because their failure modes only show up under real data volume and real consumers.

KEY TAKEAWAY A query or event schema that works in a demo with a handful of rows has told you almost nothing about whether it's actually correct.

AGENTIC TOOLS IN USE

How this week's agentic tools help across four different technologies.



Copilot for Angular Scaffolding

Drafting component classes, templates, and service injection quickly, reviewed for unsubscribe/state-management correctness before merge.



AI-Assisted SQL Drafting

Using an assistant to draft a starting query or JPA repository method, then independently running EXPLAIN to check the actual execution plan.



Agentic Assistants for Cross-Layer Tracing

A tool like Claude Code or Cursor tracing a bug from an Angular component through a REST call into a JPA repository query -- useful for following the full stack in one session.



AI-Assisted Event Schema Drafting

Using AI to draft a first version of a Kafka event record, with the versioning and compatibility decision still made deliberately by a human.

KEY TAKEAWAY This week's agentic tools are strongest at tracing a problem across layers quickly -- the performance and compatibility judgment calls at each layer still belong to you.

DISCUSSION PROMPT

A short classroom conversation to close out Week 4's AI thread.

DISCUSSION QUESTIONS

- 1) An AI assistant suggests a SQL query that returns correct results instantly in your dev database. What would make you check its query plan anyway before shipping it?
- 2) Across Angular, SQL, and Kafka this week, which layer would you trust an AI suggestion in with the least review, and which with the most -- and why?

WHY IT'S WORTH DEBATING

- Full-stack weeks are exactly when it's tempting to let AI move fastest, because there's the most ground to cover. Naming which layers deserve more scrutiny keeps that speed from becoming a liability.

FACILITATION NOTE

- If anyone in the room has actually run EXPLAIN on an AI-suggested query this week, have them share what they found -- a real example of a missing index lands better than the general warning.

KEY TAKEAWAY Speed across four technologies in one week is only a win if every layer's AI-assisted code gets the review appropriate to its actual failure mode.

WEEK 5 AI THREAD

AI in DevOps and Delivery

THIS WEEK'S DOMAIN

- Week 5 covers application security testing, containerization, Kubernetes and k3s, GitHub Actions CI/CD, and Infrastructure as Code -- the layer where code becomes running, deployed infrastructure.
- For the first time in the bootcamp, the code students write can directly create, modify, or destroy real running infrastructure.

WHY AN AI THREAD NOW

- This is the sharpest risk week in the whole bootcamp: an AI-drafted Terraform plan, Ansible playbook, Dockerfile, or GitHub Actions workflow can look completely syntactically correct while being insecure or destructive once actually applied.
- Unlike a bad merge to application code, a bad apply against real infrastructure is not always something a quick revert can undo.

WHAT'S AHEAD IN THIS THREAD

- Mandatory-review practices for AI-drafted infrastructure, AI's real impact on DevOps and delivery work, the agentic tools relevant to CI/CD and IaC, and a discussion question on the cost of an unreviewed terraform apply.

KEY TAKEAWAY Nothing AI drafts in this week's domain -- Terraform, Ansible, Dockerfiles, GitHub Actions workflows -- gets applied, run, or merged without a human reading every line first.

AI BEST PRACTICES AND GOVERNANCE

Why AI-drafted infrastructure code needs mandatory human review before it ever runs.



Mandatory Review Before Apply

An AI-drafted Terraform plan, Ansible playbook, or Kubernetes manifest gets a full human read-through before apply, run, or kubectl apply -- no exceptions, no matter how routine it looks.



Treat Destructive Suggestions as Red Flags

An AI assistant that suggests a resource replacement, a --force, or a broad DROP or DELETE in an IaC context needs extra scrutiny -- these are exactly the changes with no easy undo.



Never Embed Secrets in AI-Drafted Config

AI-drafted Dockerfiles, workflows, and manifests must pull secrets from a secrets manager or GitHub Actions secrets -- never hardcode credentials because AI suggested inline values as a placeholder.



Triage AI-Assisted Security Scan Results

AI can help summarize and prioritize a security scan's findings, but a human still decides which findings are real, which are noise, and which block a release.

KEY TAKEAWAY In DevOps and delivery, the review gate is not optional friction -- it's the only thing standing between an AI-drafted mistake and production infrastructure.

AI'S IMPACT ON DEVOPS AND DELIVERY

Where AI speeds up delivery work, and where its mistakes are hardest to undo.

IMPACT AREA

Area	What Changes This Week
GitHub Actions workflows	Fast starting YAML -- but permissions, secrets, and approval gates need a careful pass.
Terraform and Ansible	A subtly wrong resource reference can mean deletion -- always review the plan output.
Dockerfiles and containerization	Usually functional, but often skips a non-root user or a minimal base image.
Security-scan triage	Saves real time sorting findings from noise -- the human still decides what blocks release.

BOTTOM LINE

This week's tools are the most consequential in the bootcamp to get wrong, because their mistakes run against real infrastructure, not just a local test suite -- review discipline matters more here than anywhere else.

KEY TAKEAWAY An AI-drafted terraform plan that looks routine can still contain a destructive change -- read the plan output, every time, before anyone applies it.

AGENTIC TOOLS IN USE

The AI tools relevant to this week's CI/CD and infrastructure work.



AI-Assisted GitHub Actions Drafting

Using Copilot or a chat assistant to draft a starting workflow file, then hand-reviewing triggers, permissions, and secret usage before it runs.



AI-Assisted Terraform/Ansible Drafting

Drafting IaC with AI assistance, then always running terraform plan, or the Ansible check-mode equivalent, and reading the output before any real apply.



AI-Assisted Security-Scan Triage

Using AI to summarize and prioritize findings from a container or dependency scan, with a human making the final call on what blocks a merge or release.



AI-Assisted Pipeline Failure Diagnosis

Asking an assistant to help interpret a failed CI/CD run's logs -- genuinely useful for speeding up diagnosis, without skipping the step of actually fixing the root cause.

KEY TAKEAWAY These tools make drafting infrastructure and diagnosing pipelines faster -- they don't shorten the review step, which stays mandatory for anything that touches real infrastructure.

DISCUSSION PROMPT

A short classroom conversation to close out Week 5's AI thread.

DISCUSSION QUESTIONS

- 1) An AI assistant drafts a Terraform change that looks like a routine update but actually forces a resource replacement, meaning a delete-then-recreate. What review step would have caught this before apply?
- 2) Who should be allowed to run terraform apply or kubectl apply on AI-drafted infrastructure code with no second reviewer -- and why should the answer probably be 'no one'?

WHY IT'S WORTH DEBATING

- Students are close to the capstone, where they'll make real infrastructure decisions with real consequences. This is the week to make the review habit concrete and non-negotiable, before the stakes get even higher.

FACILITATION NOTE

- Walk through what a Terraform 'resource replacement' actually does step by step on the board -- many students haven't seen the distinction between an in-place update and a destroy-and-recreate until it's shown concretely.

KEY TAKEAWAY The cost of skipping a review on AI-drafted infrastructure code isn't a bug report -- it's downtime, data loss, or a security incident, which is why the review gate is mandatory, not optional.

WEEK 6 AI THREAD

AI in the Capstone

THIS WEEK'S DOMAIN

- Week 6 is the capstone: architecture planning, building the backend, frontend, and persistence layer, wiring security and CI/CD, deploying the result, and defending the whole thing in a final presentation and retrospective.
- Every layer built in Weeks 2 through 5 -- backend, frontend, persistence, security, and delivery -- comes together here as one deployed, defended system.

WHY AN AI THREAD NOW

- Every practice from the last five weeks converges here -- and for the first time, students must be able to explain and defend every AI-assisted decision they made, out loud, to a panel. Governance stops being theory and becomes something you're accountable for.
- A defense panel's questions are designed to find exactly the gap between code that works and a decision a student can actually explain.

WHAT'S AHEAD IN THIS THREAD

- AI governance in practice across a real deliverable, AI's impact on how the capstone actually gets built, the agentic tools students will lean on hardest this week, and a discussion question about defending an AI-assisted decision under questioning.

KEY TAKEAWAY The capstone is where six weeks of 'verify before you trust it' either becomes a real habit you can defend under questioning, or gets exposed as something you only did when someone was watching.

AI BEST PRACTICES AND GOVERNANCE

Accountability, not just assistance -- the standard for AI use across the capstone.



Keep an AI Usage Log

Track what you used AI for across the capstone -- which files, which decisions, which suggestions you accepted or rejected -- as a real, expected deliverable artifact, not an afterthought.



Be Ready to Defend Every AI-Assisted Line

If a panelist points at a line of AI-assisted code and asks 'why did you do it this way,' the answer has to be yours -- 'the AI suggested it' is not a defense.



Full Governance Stack Applies at Once

Every rule from the last five weeks -- no secrets in prompts, mandatory review of security and infrastructure config, verify against real docs -- applies simultaneously across a real, multi-layer deliverable now.



Escalate Judgment Calls, Don't Guess

If an AI suggestion touches a genuinely uncertain architecture or security decision under deadline pressure, ask an instructor rather than silently accepting the AI's answer to keep moving.

KEY TAKEAWAY The capstone doesn't introduce new AI rules -- it's where every rule from the last five weeks applies at once, under real accountability, for the first time.

AI'S IMPACT ON THE CAPSTONE

How AI actually shows up across a full, defended, multi-layer deliverable.

IMPACT AREA

Area	What Changes This Week
Architecture and ADR drafting	Can draft a first pass -- the trade-off reasoning has to be genuinely yours to defend.
Backend, frontend, persistence build	Every per-layer risk from Weeks 2 through 4 applies at once, across a larger deliverable.
Security, CI/CD, and deployment	Week 5's mandatory review still applies -- a capstone is real infrastructure too.
Final defense and retrospective	New this week: can you explain an AI-assisted decision under live questioning.

BOTTOM LINE

Nothing about AI's impact changes in the capstone -- what changes is that, for the first time, someone is going to ask you to account for it, out loud, in real time.

KEY TAKEAWAY The capstone is the first time AI governance gets tested under questioning instead of just under a rubric -- treat the defense as the real final exam on responsible AI use.

AGENTIC TOOLS IN USE

The full toolkit from the bootcamp, now applied to one real, defended deliverable.



Copilot Across the Whole Codebase

Inline suggestions and chat-based scaffolding across backend, frontend, and persistence code -- reviewed with the same discipline built since Week 2.



Agentic Coding Assistants for Multi-File Work

A tool like Claude Code or Cursor for tracing and fixing issues that span the full stack -- genuinely valuable at capstone scale, where manual tracing is slow.



AI-Assisted Infrastructure and CI/CD Drafting

Drafting the capstone's GitHub Actions workflows and any IaC with AI assistance, under the same mandatory human-review gate from Week 5.



AI-Assisted Documentation and Retrospective Drafting

Using AI to draft a first pass of the capstone's documentation or retrospective notes, then rewriting it in the student's own understanding, since the defense will test exactly that.

KEY TAKEAWAY Every agentic tool from the bootcamp converges on the capstone -- the differentiator is no longer which tools you used, but whether you can account for every decision they helped you make.

DISCUSSION PROMPT

A short classroom conversation to close out Week 6's AI thread -- and the AI Threads series.

DISCUSSION QUESTIONS

- 1) A defense panelist asks you to explain why your AI-assisted code makes a specific security or architecture choice, and you genuinely don't remember why -- you accepted the suggestion weeks ago. What does that moment tell you about how you used AI during the build?
- 2) Looking back across all six weeks, which AI Thread's discipline do you think you'll actually keep using after the bootcamp, and which one do you think you'll let slip -- and why?

WHY IT'S WORTH DEBATING

- This is the honest, retrospective version of everything the AI Threads series has been building toward -- real accountability is easiest to talk about in the abstract and hardest to actually practice under a defense panel's questions.

FACILITATION NOTE

- Let this run longer than earlier discussion prompts -- it's the closing conversation of the whole series. Encourage specific, personal answers over general statements; the second question especially rewards honesty over the 'right' answer.

KEY TAKEAWAY Six weeks of AI Threads come down to one test: can you explain, out loud and under real questioning, exactly what you built and why -- with or without AI's help drafting it.

"AI can draft the first pass -- only you can be accountable for what ships."

AI THREADS COMPLETE

AI Threads

Six Weeks of Disciplined Practice

You've carried the same three lenses -- best practices and governance, this week's technical impact, and the agentic tools in play -- across Java fundamentals, backend testing, Spring, the full stack, DevOps, and the capstone. Six threads, one discipline: verify before you trust it, and be ready to defend every AI-assisted decision you ship.